

Guidelines
for completing term papers in the course
“Object-Oriented Programming” for students
majoring in
121 “Software Engineering”

Ministry of Education and Science of Ukraine
Vinnytsia National Technical University

Guidelines
for completing course assignments in the course
“Object-Oriented Programming” for students
majoring in
121 “Software Engineering”

Vinnytsia
VNTU
2023

Recommended for publication by the Methodological Council of Vinnytsia National Technical University of the Ministry of Education and Science of Ukraine (Minutes No. 5 dated December 22, 2022)

Reviewers:

D. H. Shtofel, Candidate of Technical Sciences, Associate Professor
V. O. Denisyuk, Candidate of Technical Sciences, Associate Professor
I. R. Arsenyuk, Candidate of Technical Sciences, Associate Professor

Methodological Guidelines for Completing Course Projects in the Discipline
“Object-Oriented Programming” for students majoring in 121
“Software Engineering” / Compiled by: D. I.
Katelnikov, A. V. Denisyuk. – Vinnytsia: VNTU, 2022. – 44 pp.

These guidelines are intended to assist students in completing term papers for the course “Object-Oriented Programming” and in preparing the accompanying explanatory note. A list of possible topics for development is provided; requirements for the software tool that will fulfill the main objective of the project are outlined; recommendations for the proper formatting of the explanatory note are given; and criteria for evaluating course projects are presented.

TABLE OF CONTENTS

1	BASIC REQUIREMENTS FOR THE COURSE PROJECT	4
1.1	Purpose and Objectives of the Course Project	4
1.2	Topic of the Course Paper	5
1.3	Requirements for Software Development	7
2	FORMAT OF THE EXPLANATORY NOTE	8
2.1	General Formatting Requirements	8
2.2	Structure of the Explanatory Note	11
2.3	Introduction to the Explanatory Note	11
2.4	Content and Format of the Main Body	13
2.5	Formatting of the Appendices	19
2.6	Formatting of the Illustrative Section	20
3	SCHEDULE FOR COMPLETION OF COURSE PROJECT AND PROCEDURE	21
	DEFENSE	
3.1	Schedule for Completion	21
3.2	Evaluation of the Course Paper	22
	LIST OF RECOMMENDED READING	24
	APPENDIX A. Course Project Assignment Options	26
	APPENDIX B. Example of a Title Page Format	34
	APPENDIX C. Example of an Individual Assignment Format	35
	APPENDIX D. Use of Algorithm Diagrams, Programs, and Data	36
	APPENDIX E. Examples of UML Diagrams	42

1 BASIC REQUIREMENTS FOR THE COURSE PROJECT

1.1 Purpose and Objectives of the Course Paper

The course paper in the discipline “Object-Oriented Programming” is designed to provide students with basic knowledge and practical skills in the fundamentals of modern technology for creating complex software products based on the ideas and principles of the object-oriented method. This knowledge is intended for use in the development of information technology software, taking into account modern requirements regarding the reliability, interface quality, and efficiency of the software products being created. The knowledge and practical skills acquired should serve as a foundation for students to master new programming systems in the future, which are based on the concepts of visual programming, CASE technologies, artificial intelligence, and so on.

While completing this course project, students should apply the knowledge they have gained in the courses “Fundamentals of Programming” and “Discrete Mathematics for Computer Science.”

Competencies that students should acquire as a result of completing this course project.

ZK02. Ability to apply knowledge in practical situations.

SK02. Ability to participate in software design, including modeling (formal description) of its structure, behavior, and operational processes.

SK03. The ability to develop architectures, modules, and components of software systems.

SK04. The ability to formulate and ensure software quality requirements in accordance with specifications, technical requirements, and standards.

SK07. Knowledge of information models for data and systems, and the ability to develop software for storing, retrieving, and processing data.

SK08. Ability to apply and develop fundamental and interdisciplinary knowledge to successfully solve software engineering problems.

SK11. Ability to implement the phases and iterations of the software systems and information technology life cycle based on relevant software development models and approaches.

SK13. The ability to make informed choices and master tools for software development and maintenance.

SK14. Ability to think algorithmically and logically.

Program Learning Outcomes.

LR03. Know the main processes, phases, and iterations of the software life cycle.

LR06. Ability to select and apply the appropriate software development methodology for a given task.

PN07. Know and apply in practice the fundamental concepts, paradigms, and basic principles of the functioning of language, tool, and computational resources in software engineering.

RN12. Apply effective approaches to software design in practice.

RN13. Know and apply methods for developing algorithms, designing software, and constructing data and knowledge structures.

RN15. Make informed choices regarding programming languages and development technologies to solve problems related to software creation and maintenance.

RN17. Be able to apply methods of component-based software development.

Assignments for term papers are selected by the instructor from a general list of topics approved by the Department of Software Engineering. Students are encouraged to propose their own topics for term papers—in consultation with the instructor—that fall outside the list provided in the course guidelines.

The purpose of individual assignments is to reinforce theoretical and practical skills in working with modern object-oriented programming technologies through the use of modern programming languages and technologies.

1.2 Topics for the Course Paper

The content of the course project corresponds to the curriculum and syllabus of the “Object-Oriented Programming” course and is determined by the assignment, which is issued no later than six days after the start of the semester during a consultation with the instructor for each student.

The course project involves several sequential stages, which must include a substantive formulation of the problem, the development of individual and technical specifications, object-oriented analysis and design of the problem’s domain with the construction of corresponding UML diagrams, the development of both the classes and objects themselves as well as the means for their instantiation, serialization/deserialization, and synchronization, and the construction of an intuitive graphical user interface for the program.

A tentative schedule for completing the course project is provided in the following sections. Students are encouraged to follow this schedule, beginning work on the project in the first weeks of the semester. During

the time of the ongoing assessments in the course “Object-Oriented Programming,” the quality, engagement, and effectiveness of students’ work will be evaluated with corresponding points, which will influence the overall grade for the course module.

The topic of the course project is related to the students’ future major. The program implementation for this course project requires students to master the following mechanisms of object-oriented programming:

1. Object Structure
2. Encapsulation
3. Messages
4. Classes
5. Inheritance
- b. Class Inheritance
7. Method Overloading
8. Access Modifiers
9. Event Handling
10. Object creation, constructors, destructors
11. String Manipulation
12. Packages and Importing Them
13. Interfaces and their implementation
14. Exceptions and their implementation
15. Identifiers and data types
16. Branching Operators
17. Loop Statements
18. Arrays
19. Creating User Interface Components
20. Working with Windows
21. Working with Numeric Data Types
22. Designing Programs as a Class Hierarchy
23. Type Conversion Between Classes
24. Destroying Objects
25. Principles of How Programming Language Compilers Work
26. Boolean data types and their processing

In addition, the course project may involve the implementation of test programs or the development of laboratory exercises for this discipline and for other disciplines related to software engineering.

The goal of the design should be to select a method of object-oriented decomposition of the domain that not only ensures the complete reproduction of the desired functionality but is also optimal in terms of programming time. Thus, when choosing a method of object-oriented

decomposition of the subject domain, an individualized approach in each specific case is essential.

After selecting a topic for their term paper, students must have a clear understanding of the system users' requirements regarding the system's functionality and the usability of its interface.

Individual course project topics are assigned by the instructor from a general list of course project topics. Students are encouraged to propose their own course project topics—in consultation with the instructor—that fall outside the list provided in the course guidelines. The independent selection of a subject area allows for an assessment of the student's level of creative activity, their ability to independently conduct a preliminary analysis of the subject area, set a specific goal for themselves, and effectively achieve it.

1.3 Requirements for Software Development

The result of the course project should generally be a fully-fledged software application for a specific operating system (Windows, Unix, Android, iOS) that implements the required functionality.

The project involves the following stages: *a preliminary design*, during which general decisions are made regarding the selection of development tools, the creation of models, and the definition of how the product will interact with its environment (users, hardware); *a technical design*, during which specific decisions are made regarding the detailing of algorithms and the documentation of algorithms and interfaces; *the implementation phase*, during which the program is coded. Based on the technical specifications and an analysis of the requirements for the technical and operational characteristics of the development object, the design technology must be determined. After selecting the design technology, the development tools for creating the project must be identified. These include a programming language, a translator, libraries, and a development environment that meet modern programming language requirements; for example, if Java is chosen, the compiler must support the latest version of the JDK—Java Development Kit (as of 2017–2018, this is JDK 8). In recent years, a large number of libraries have been created for every programming language. Even the standard Java library consists of several thousand classes that allow for solving a wide variety of tasks. As for the development environment, it should be intuitive, user-friendly, and fast. In fact, the standard development environments for the Java programming language are NetBeans IDE—a free, open-source environment—and IntelliJ IDEA—a feature-rich commercial environment that is, however, free for use in educational institutions.

2 FORMATTING THE EXPLANATORY NOTE

2.1 General Formatting Requirements

When formatting the explanatory note, you must comply with the following requirements of DSTU 3008:2015. The text of the explanatory note must be typed on a computer and printed.

Font and Indentation

The text of the explanatory note must be typed in any word processor using the *Times New Roman* font, size 14, with a line spacing of 1.15–1.5. Margins: left—2.5 cm, right—1 cm, top and bottom—1.5 cm. The font and line spacing in the appendices may be chosen at the author's discretion, provided they are legible and understandable.

Page Numbering

Pages must be numbered, starting with the table of contents in the upper right corner of the page. The title page, pages with the abstract, and the page with the individual assignment are not numbered.

The numbering of appendices continues the main numbering sequence.

Formatting of Sections and Subsections

The structural elements of the main body of the explanatory note are sections, subsections, paragraphs, subparagraphs, and lists.

A section is the main level of text division; it is numbered and has a heading. A subsection is a part of a section; it is numbered and has a heading. A paragraph (subparagraph) is a part of a section or subsection (paragraph); it is numbered and may have a heading.

Sections are numbered sequentially (using only Arabic numerals) throughout the entire document (1, 2, etc.). Subsections are numbered within each section, and paragraphs are numbered within each subsection in the following format (3.1, 3.2, 3.2.1, 3.2.2.1, etc.). The numbers indicating the section or subsection should not extend beyond the margin. Do not place a period after the number; instead, leave a single space.

It is recommended that each section begin on a new page. The section heading (Level 1 heading) is centered in uppercase letters with a slightly bolder font.

Headings of subsections, paragraphs, and subparagraphs (if a heading is present) are written in lowercase letters, starting with a capital letter. One line is skipped before and after the heading, or an 18-point spacing is used. Second-level headings are written in a bolder font.

It is permissible to place text between section and subsection headings, as well as between subsection and paragraph headings.

Lists

Lists, if necessary, may be included within paragraphs or subparagraphs. A colon is placed before the list. Each item in the list should be preceded by a lowercase letter of the Ukrainian alphabet in parentheses, or—if unnumbered—a hyphen (first level of detail). For example,

The main problems are as follows:

- industrial espionage;
- theft and copying;
- unauthorized modification;
- piracy.

Arabic numerals in parentheses are used for further detail.

Table Formatting

A table is placed symmetrically relative to the text after the first reference on that page or on the next page if it does not fit on the current page, and in such a way that it is convenient to view without rotation or with a 90° rotation. Tables in the text of the explanatory note are typeset in the main font; in some cases, the font size may be reduced to 10–12. Table captions are placed above the table, indicating its number and title, and aligned to the left margin. For example:

Table 1.1 – Concepts of Environmental Protection

Concept	Description
1	2
1. Class	A specific abstract entity, such as “Dog,” “Cat,” “Car,” “Product Price,” which, at the programming level, represented by variables (data fields) and methods that operate on these data fields
2. Object	A specific instance of a class. For example, your neighbor’s green “Toyota” is an instance of the “Car” class. Essentially, it is a class whose fields have been initialized and which has been loaded into the computer’s memory. Based on a single class, you can create many objects

All tables must be referenced in the form “... in Table 1.1” or in parentheses within the text (Table 1.1). References to a previously cited table are made using the abbreviation “see” (see Table 1.1) either in the flow of the text or at the end of the sentence.

When parts of a table are carried over to other pages, the column headings are repeated or continued. It is permissible to number the columns at the beginning of the table and, when carrying parts of the table over to subsequent pages, to repeat only the column numbers. In all cases, the title (if present) of the table is placed only above

the first part, and above the other parts, on the left, write “Continued from Table 1” without a period at the end, for example:

Continued from Table 1.1

1	2
3. Inheritance	Creating new classes based on others
4. Interface	A reference data type. Interfaces are similar to classes, but their data fields are constants and their methods are not implemented. Objects are not created based on interfaces, but classes can implement a specific interface. And through an object variable of the interface type, you can call the implementations of those methods

Formatting Figures

Figures are placed in the text or in the appendices. In the text, a figure is placed opposite the text after the first reference to it or on the next page if it does not fit on the current page without being rotated. All figures must be referenced in the following format: “... in Fig. 3.6,” or in parentheses within the text (Fig. 3.6). References to a previously cited figure are made using the abbreviation “see” (see Fig. 3.4) either in the flow of the sentence or at the end of the sentence.

Each figure must have a number and a caption, centered below the figure. Do not place a period at the end, and do not use a hyphen. If the figure’s title is long, continue it on the next line of the caption, starting with the title. For example:



Figure 3.6 – Parameter settings window Leave

one blank line between the illustration and the text.

Illustrations are numbered within each section, indicating the section number and the sequential number of the illustration within the section, separated by a period. Numbering throughout the entire document is permitted.

Formula Formatting

Each formula is written on a new line, aligned with the text, in italics. Leave one blank line between the formula and the text.

Letter symbols (variables) in a formula must conform to established conventions. Their definitions are provided in the text or immediately below the formula. To do this, place a comma after the formula and list the definitions for each symbol on a new line in the order in which they appear in the formula, separated by a semicolon. The first line must begin with a paragraph containing the word “where” and no punctuation mark following it.

All formulas within a section are numbered using Arabic numerals. The number is placed in parentheses on the right side, at the end of the line, aligned with the end of the formula. The formula number consists of the section number and the sequential number of the formula within the section, separated by a period. Numbering throughout the entire document is permitted.

2.2 Structure of the Explanatory Note

Each stage of the course project must be reflected in the explanatory note. The explanatory note must correspond to the individual assignment, and its format must comply with current standards in effect at the time of the project’s completion, including any official amendments that have been enacted.

The explanatory note for a term paper must have the following structure:

- introduction;
- main body;
- appendices;
- illustrative section.

2.3 Introduction to the Explanatory Note

The introductory section must contain:

- title page;
- individual assignment;
- an abstract in two languages;
- a table of contents.

Title Page

The title page is the first page of the term paper and is not numbered. The title page must follow the established format. A sample title page is provided in Appendix B.

Individual Assignment

The specific content of each term paper and the stages of its completion are determined by the advisor based on the individual assignment approved by the department chair.

Prior to this, the advisor issues the individual assignment for the term paper

. The individual assignment is not included in the table of contents and must be the second page following the title page. A sample of the individual assignment for the term paper is provided in Appendix C.

The course paper advisor proposes a list of the main stages for carrying out the course paper assignment, which is typically standard and provided in the developed methodological guidelines. This list must be included in the individual assignment. In addition, the individual assignment must specify the project's initial and final data.

Depending on the specifics of the discipline, the course project advisor may propose a topic that requires specific justification and the development of a specific individual assignment. The individual assignment for the course project must include the date of issuance and the signatures of the advisor and the student.

The course paper assignment must be prepared by the student no later than the second week of the academic semester and signed by the instructor who issued the assignment and the student who accepted it for completion.

Abstract

The abstract is intended to provide an overview of the text of the term paper. It should briefly describe the purpose of the paper, the methods used to achieve the stated objective, and a summary of the results obtained. The abstract should be approximately one-third of a page in length.

The abstract is placed immediately after the page with the individual assignment, starting on a new page that is not numbered. The title ("ABSTRACT") is centered on the page, followed by a single blank line.

Table of Contents

The table of contents is placed immediately after the abstract, starting on a new page. The table of contents includes: the introduction; a sequential list of the titles of all chapters, sections, subsections, and sub-subsections (if they have headings) of the paper; conclusions; a list of references; and the titles of appendices and the page numbers where the material begins.

The table of contents does not include the title page, the individual course assignment, or the abstract. Numbering in the table of contents begins with the introduction (in accordance with the numbering in the explanatory note). The table of contents itself is the fourth page according to the numbering in the explanatory note. Page numbering must be continuous, including the appendices.

The titles in the table of contents must correspond exactly to the titles in the explanatory note as they appear in the text. It is recommended to generate the table of contents automatically within the text document using the features of your chosen word processor.

Example of a table of contents:

INTRODUCTION

1 DEVELOPMENT ...

1.1 Options ...

1.1.1 ...

.....

2 TITLE OF THE SECOND CHAPTER

2.1 Subsection Headings

2.1.1 ...

.....

.....

CONCLUSIONS

LIST OF REFERENCES APPENDICES

APPENDIX A. Title of the first appendix **APPENDIX**

B. Title of the second appendix

.....

2.4 Content and Format of the Main Body

The main body consists of the following components:

- an introduction;
- an analysis of the current state of the issue and a justification of the topic;
- implementation of the course paper’s objective;
- testing of the developed solution;
- conclusions;
- a list of references.

2.4.1 Introduction

Introduction is written from a new numbered page
with the heading

“INTRODUCTION” centered in uppercase letters.

The introduction should be brief and address the relevance, significance, current status, and purpose of the term paper. In the introduction and throughout the text, the use of abbreviations and acronyms is not permitted, except for those that are generally accepted. If it is necessary to use certain generally accepted abbreviations (acronyms) in the text, the abbreviation must be indicated in parentheses the first time it appears. Only after that may the abbreviation be used throughout the text. For example, object-oriented programming (OOP). It is not recommended to use abbreviations in the titles of chapters, sections, paragraphs, and subparagraphs.

The introduction should briefly address the following issues:

- the current state of the problem in the field to which the development relates;
- the field of application and purpose of this development;
- *the goal* and general formulation of *the problem*;

– *the relevance*, which should be presented in the last paragraph of the introduction to briefly outline the essence of the selected project.

The introduction should not exceed 1–2 pages.

2.4.2 Analysis of the Current State of the Issue and Justification of the Topic

This section is mandatory and involves a review and analysis of known analogues and existing software products in the relevant field, etc. This section should include a comparative analysis of existing similar software programs used to solve analogous problems, as well as a brief analysis of **domestic** and foreign **patent** and **scientific** literature from the past few years, and electronic documents (e-books, websites, web pages). References to sources within the text of the explanatory note are mandatory and are cited in square brackets, indicating the sequential number in the list.

The recommended length of this section is 4–5 pages.

2.4.3 Implementation of the Main Objective of the Course Paper

This section should describe in detail the steps necessary to achieve the project's objective, the requirements for the development work, and the expected results.

Thus, this section should include the following stages:

1. Development of the program interface and usage examples. At this stage, both the general program interface and usage examples are created to illustrate the main ways in which objects interact with the user.
2. Development of class, object, and state diagrams. At this stage, the results of decomposing the domain into classes and objects are documented in the form of inheritance, aggregation, composition, interaction, sequence, and state diagrams.
3. Development of the graphical display subsystem. At this stage, images of micro- and macro-objects are created, and graphical special effects are generated as needed.
4. Use of JavaFX programming tools. At this stage, specific issues related to object synchronization, message passing, and so on are addressed.
5. Development of the data serialization/deserialization subsystem. At this stage, methods for saving the program's data structure to external storage media are developed.
6. User Manual. At this stage, a user manual is developed explaining how to install and run the program, as well as how to manage objects while the program is running.

When drafting the text of the explanatory note, it is prohibited to copy material from literary sources or scan figures that are

to the main body of the paper. It is recommended to place some of the descriptive material (in the form of tables, figures, or graphical information) in the appendices to the explanatory note and to refer to these appendices in the text.

If the course project is to result in a software product, it is recommended to describe the development of the program's general operational scheme or flowchart in separate subsections. For individual modules that will be part of the software, it is necessary to develop flowcharts of algorithms, structural diagrams showing the main blocks, step-by-step descriptions of the algorithm with branching, etc.

The inclusion of diagrams in the explanatory note is mandatory. Their number and types should be selected by the developer in such a way that they help to understand the operation of the program as a whole and its components. All diagrams must be prepared in accordance with the requirements of current standards (Section 2.6).

When describing the software implementation of the problem, provide a step-by-step description of the implementation of the main algorithm for the given problem. The main programming language structures used in this work are described. Code snippets (5–10 statements, not the entire program!) are provided. The main variables, auxiliary modules, classes, procedures, and functions, as well as the main and header files used in software development, are described.

If you have developed your own subroutines or functions, they must be described and include references to the appendices where the program listing is located. If standard functions are used, explain their purpose and rationale.

This section should be one of the most specific and substantive and should occupy approximately two-thirds of the explanatory note (13–15 pages).

2.4.4 Use of Schematics and Diagrams

It is recommended to develop and describe various diagrams (Appendix D) as separate subsections of the main section or at the end of specific subsections:

- data diagrams;
- program flowcharts;
- system operation diagrams;
- program interaction diagrams;
- system resource diagrams.

The program developer must decide for themselves which diagrams are appropriate to develop in their work.

Diagrams can be created as UML diagrams, which may include the following (Appendix D):

- use case diagram;
- class diagram;
- state diagram;
- activity diagram;
- sequence diagram;

- component diagram, etc.

Schemes and diagrams can be used at various levels of detail, with the number of levels depending on the size and complexity of the data processing task.

Standard software tools specifically designed for this purpose or built-in tools of the software environments in which the programs are implemented can be used to construct diagrams and charts.

2.4.5 Program Testing and Development of Instructions

This section may be either a standalone section or a subsection of the main section of the explanatory note. It should be devoted to testing the developed program for correct operation, ease of presenting materials, etc., as well as to formulating instructions and recommendations for working with it.

Here, it is necessary to demonstrate the entire execution process of the program across all its operating modes. It is recommended to include a subsection titled “Development of Test Cases” in the explanatory note, in which various sets of input data should be prepared: both valid and those that cause errors. When conducting and describing the analysis of the program’s operation, you may include figures that illustrate the program’s operation (screenshots) and demonstrate its main modes and functional capabilities. These illustrations may be included in the appendices, with references to them provided in the text of the explanatory note.

If the topic of this term paper is the development of a software tool, you must include specific instructional materials on how to use the program. That is, you should develop guidelines for using the program: instructions for the programmer, the systems programmer, the operator (user), and a maintenance manual. Instructions for using the program are included in the appendices.

Maintenance Manual

The maintenance manual must contain the following sections:

- *introduction* (purpose of the manual, list of operational documents that must be used in addition to the manual during maintenance and operation);
- *general guidelines* (maintenance procedures, guidelines on the organization and specifics of maintenance);
- *requirements for technical resources* (specifying the minimum set of technical resources required to ensure the software’s operation);
- *Description of functions* (a comprehensive list of functions performed by this program; a description of the interoperability between hardware and the program, including the error-handling method; a description of the organization of input and output data for verifying operational capability; a description

device interactions with the program, the results of these interactions, and conclusions drawn from the program's performance).

System Programmer's Manual

The system programmer's manual must include the following sections:

- *general information about the program* (the program's purpose and functions, and information about the hardware and software that enable the program to run);
- *program structure* (information about the structure of the software product, its components, the relationships between components, and the relationships with other programs). A diagram showing the relationships between programs is appropriate here;
- *program configuration* (a description of the steps for configuring the program for specific application conditions—configuration for the available hardware, selection of functions, etc. Explanatory examples should be provided if necessary);
- *program verification* (a description of verification methods that allow for a general conclusion regarding the program's operability—test cases, execution methods, and results);
- *messages to the system programmer* (this section should include the text of messages that may appear during the configuration of the security system or the verification of the program's correct operation, along with a description of their content and the actions that must be taken in response to these messages).

Programmer's Manual

The programmer's manual must include the following sections:

- *purpose and conditions of use of the program* (this section should specify the purpose and functions performed by the program, as well as the conditions required for the program to run—amount of RAM, requirements for device configuration and parameters, software requirements, etc.);
- *program characteristics* (a description of the program's main characteristics and features—performance metrics, operating modes, means of verifying correct execution and program self-recovery, etc., the development environment, and the ability to make adjustments);
- *program invocation* (a description of the procedures for invoking the program—methods of control transfer, parameters, etc.);
- *input and output data* (a description of the organization of the input and output information used and, if necessary, its encoding);
- *messages* (texts of messages displayed to the programmer or operator during program execution, a description of their content, and the actions to be taken in response to these messages).

Operator Manual

The operator's manual must include the following sections:

- *purpose of the program* (information about the program's purpose and

- information sufficient to understand the program's functions and its operation);
- *program execution requirements* (conditions necessary for the program to run: minimum and/or maximum hardware and software requirements, etc.);
 - *program execution* (the sequence of operator actions required to load, start, execute, and terminate the program; a description of the functions, format, and possible variations of the commands used by the operator to load and control the program's execution);
 - *messages to the operator* (the text of messages displayed during program execution, a description of their content, and the corresponding actions the operator should take: in the event of a failure, the option to restart the program, etc.).

2.4.6 Conclusions

Conclusions are presented on a new, numbered page, beginning with the word "CONCLUSIONS" centered in uppercase letters, followed by a single blank line. The conclusions present the main results of the coursework. The stages of implementing the coursework task are briefly described according to the main sections. Based on the research conducted, well-founded conclusions are presented regarding the object-class decomposition performed, the strengths and weaknesses of the development, the difficulties encountered during program development and their underlying causes, and possible ways to overcome them. It is essential to provide recommendations for the practical application of the developed software tool, as well as ways (prospects) for improving the developed software.

2.4.7 List of References

The list of references should be formatted on a new, numbered page, beginning with the words "LIST OF REFERENCES" centered in uppercase letters, followed by a single blank line.

The list contains the sources used in the course of the work, to which references must be made in the text of the explanatory note. Sources (books, articles, patents, journals, websites) are listed in the general list in the order in which they are cited in the text. References to sources are indicated in square brackets [...], specifying the sequential number in the list.

Each source must be formatted in accordance with DSTU 8302:2015 [2]. The publisher, year of publication, and number of pages must be specified. Sources are listed in their original language. In the list, each source begins on a new line and is numbered with Arabic numerals, starting with one.

Example of a list of references of various types:

Example of a citation for a book:

1. Romanyuk, O. N., Katelnikov, D. I., & Kosovets, O. P. Web Design and Computer Graphics: Textbook. Ministry of Education and Science of Ukraine. Vinnitsa National Technical University. Vinnitsa: VNTU, 2007. 142 pp.: ill.
2. Bulgakova, O. S., Zosimov, V. V., Bronitska, N. A., & Tankova, N. V. Computer Science: Visual Programming: A Teaching and Methodological Guide. Ministry of Education and Science of Ukraine, V. O. Sukhomlynsky National University. Kherson: Oldi-Plus, 2014. 312 pp. ISBN 978-966-289-039-6.

Example of a citation for an article in a periodical:

3. Gaev, E.O., Rozhok, O., and Ovcharchin, N. "Sound and Music in the Programming Course." *Software Engineering*. 2014. No. 3. pp. 41–48.

Example of a citation for standards:

4. DSTU 3008:2015. Information and Documentation. Reports in the Field of Science and Technology. Structure and Formatting Rules. Official publication. Replaces DSTU 3008-95; effective as of July 1, 2017. Kyiv: State Enterprise "UkrNDNC," 2016. 26 pp. (National Standard of Ukraine).

Example of a patent citation:

5. Direct current amplifier: Pat. 136106 Ukraine: IPC H03F 3/26 (2006/1). G05B 1/00 / O. D. Azarov, V. A. Garnaga, O. O. Gudymenko (Ukraine); applicant and patent owner: Vinnytsia National Technical University. No. u201812835; filed Dec. 26, 2018; published Aug. 12, 2019, Bulletin No. 15. 6 pp.: drawings.

Example of a citation to electronic resources:

6. Vinnitsa National Technical University: website. URL: <https://vntu.edu.ua/> (accessed October 7, 2022).
7. Bublyk, V. V. Object-Oriented Programming: A Textbook / V. V. Bublyk. Kyiv: IT-Knyha, 2015. 624 pp. URL: <http://surl.li/gxfzg> (accessed November 10, 2022).

2.5 Formatting of Appendices

Appendices should contain material that:

- is necessary for the completeness of the report, but including it in the main body of the report could disrupt the organized and logical presentation of the work;
- cannot be sequentially placed in the main body of the report due to its large volume or the methods of presentation;
- may be omitted for a general audience but is necessary for specialists in the field.

Appendices may include:

- additional illustrations, diagrams, or tables;
- materials that, due to their large volume, specific presentation, or format

cannot be included in the main body (original photographs, intermediate mathematical proofs, disassembled code listings; test reports; instructions; methodologies; descriptions of computer programs developed or used in the course of the work, etc.);

- instructions for using the program.

Each appendix must begin on a new page, with the word “APPENDIX” centered at the top of the page, followed by a space and its designation. Appendices are numbered sequentially using uppercase Ukrainian letters, with the exception of the letters І, Є, З, І, Ї, О, Ч, and Ї; for example, **APPENDIX A, APPENDIX B**, etc. If there are more appendices than there are letters, the numbering continues with Arabic numerals. It is permissible to label appendices with Latin letters, except for *I* and *O*.

Each appendix must have a thematic (descriptive) title, which is written in the middle of the line in lowercase letters, starting with a capital letter.

The pages of the appendices are numbered, continuing the general numbering from the explanatory note.

All appendices include	in table of contents,	indicating
the number,	the title	and the pages on which they begin.

2.6 Formatting the Illustrative Section

The illustrative section is intended to help the student successfully defend their work. The illustrative material should include all materials that help present the stages of the course project’s development, key diagrams, algorithms, the mathematical model, and fragments of the developed program’s interface.

Before the illustrative section, a separate sheet is inserted into the explanatory note, with the heading

“Illustrative Section,” followed directly by the pages of illustrative material. Each page should be formatted as a poster: the caption should be centered at the top, followed by the main content of the poster centered below it.

The sheets of the illustrative section are not numbered and are not included in the table of contents of the explanatory note.

3 SCHEDULE FOR COMPLETING THE COURSE PAPER AND PROCEDURE FOR DEFENSE

3.1 Schedule

The following schedule for completing the course paper is recommended, taking into account students' independent work during the 2nd trimester (18 weeks) for full-time students and the 6th semester for part-time students.

Section Contents	Deadline
Receiving the coursework assignment, developing and formatting the individual assignment	1–2 weeks
Analysis of literature and the current state of the problem; justification of the choice of an object-oriented programming language	2–3 weeks
Development of the software structure: understanding the algorithms for subtasks, solving test examples, developing the interface, justifying the need for additional tools, developing the structure of input and output data, selecting the necessary software tools, etc.	3–5 weeks
Software development and debugging: programming and testing of main procedures and functions, software implementation of the interface, software implementation of file handling and control elements, implementation of data integrity checks, etc.	6–11 weeks
Testing the development and fixing identified defects. Preparing test cases, verifying the completeness of the implemented functionality	9 weeks
Preparation of an explanatory note for the course project; development of recommendations for operating the developed program	10–11 weeks
Submission of the course project for preliminary review: demonstration of the program's operation and a draft of the explanatory note	12 weeks
Revising and supplementing (if necessary) the program based on feedback from the course project advisor; incorporating feedback and revising the explanatory note	Weeks 13–16
Defense of the course project	Weeks 17–18

The supervisor determines whether the course project is ready for defense based on the results of a preliminary review of the quality of the explanatory note and the feasibility of the program. The explanatory note must be submitted to the advisor for review at least one week before the scheduled defense date. If the project is complete and contains no fundamental errors, the advisor admits the student to the defense. Otherwise, the project is returned to the student for revision. After receiving a positive assessment of the project's readiness, the student must defend it before a committee consisting of

two faculty members appointed by the department.

3.2 Assessment of the Course Paper

After receiving a positive assessment confirming the readiness of the course paper, the student must defend it before a committee consisting of two faculty members appointed by the department.

Course papers are graded on a 100-point scale:

Total points for all components of the course	ECTS Grade	Grade on the national scale
90–100	A	Excellent
82–89	B	Good
75–81	C	
64–74	D	
60–63	E	satisfactory
35–59	FX	Unsatisfactory, with the possibility of retake
0–34	F	Fail with mandatory retake of the course

Grading scale for course assignments:

Explanatory Note	Practical implementation	Illustrative section	Instructions	Defense of the work	Total points
up to 20	up to 45	up to 10	10	up to 15	100

The Explanatory Note (EN) is evaluated based on the following criteria:

- compliance with the requirements for the format of the Explanatory Note in accordance with DSTU 3008:2015;
- alignment of the Explanatory Note and the program with the individual assignment;
- consistency of the explanatory note with the program, which is provided for defense;
- the logical coherence of the sections of the explanatory note.

The program component is evaluated based on the following criteria:

- fulfillment of the main objective of the term paper;
- clarity and user-friendliness of the program interface;
- the presentation itself, and compatibility with standard software;
- demonstration of the completeness of the implemented functionality of the program through this development;
- the presence of user manuals in the applications and their consistency with the software.

The defense of the project is evaluated based on the following criteria:

- demonstration of a fully functional software tool or other

- development deliverables;
- the ability to explain the key aspects of the development;
 - well-reasoned answers to questions from committee members;
 - presentation of the project (presentation, illustrative material, etc.).

LIST OF RECOMMENDED READING

1. Bublyk, V. V. Object-Oriented Programming: Textbook. Kyiv: IT-Book, 2015. 624 pp.

2. Bibliographic reference. General Provisions and Rules for Preparation: DSTU 8302:2015 / National Standard of Ukraine. Official Edition [First published; effective as of July 1, 2016]. Kyiv: State Enterprise "UkrNDNC," 2016. 17 pp. (Information and Documentation).

3. Katelnikov, D. I., & Piletsky, V. D. Implementation of New Java Technologies in the Educational Process // Electronic Information Resources: Creation, Use, Access: Proceedings of the International Scientific and Practical Internet Conference, November 9–10, 2021. Sumy/Vinnytsia:

NIKO/VNTU, 2021. pp. 74– URL:

https://ir.lib.vntu.edu.ua/bitstream/handle/123456789/34859/%d0%9c%d0%b0%d0%ba%d0%b5%d1%82_74-76.pdf?sequence=1&isAllowed=y

(acce

ssed: September 12, 2022).

4. Methodological Guidelines for Laboratory Work in the Course "Object-Oriented Programming" / eds.: D. I. Katelnikov, A. V. Denisyuk. Vinnytsia: VNTU, 2018. 45 pp.

5. Methodological Guidelines for Students' Independent Work on Designing UML Diagrams as Part of Course Projects in the Course

"Object-Oriented Programming" for students majoring in 121

"Software Engineering" (full-time and part-time programs) / eds.: D. I. Katelnikov, O. O. Dudnik, A. V. Denisyuk. Vinnytsia: VNTU, 2020. 28 pp.

URL:

<https://ir.lib.vntu.edu.ua/bitstream/handle/123456789/34684/88707.pdf?sequence=2&isAllowed=y> (accessed September 7, 2022).

6. Bruce Eckel. Thinking in Java. 4th Edition. London: Pearson, 2006. 1,150 pp. URL: https://sd.blackball.lv/library/thinking_in_java_4th_edition.pdf (accessed October 11, 2022).

7. Kathy Sierra, Bert Bates, Trisha Gee. Head First Java: A Brain-Friendly Guide. 3rd Edition. O'Reilly Media, 2022. 752 pp.

8. Marc Loy, Patrick Niemeyer, Daniel Leuck. Learning Java: An Introduction to Real-World Programming with Java. 5th Edition. O'Reilly Media, 2020.

520 pp.

URL:

[http://repository.itb-ad.ac.id/157/1/761.%20Learning%20Java%20An%20Introduction%20to%20Real-World%20Programming%20with%20Java%](http://repository.itb-ad.ac.id/157/1/761.%20Learning%20Java%20An%20Introduction%20to%20Real-World%20Programming%20with%20Java%20) (accessed October 11, 2022).

9. UML Profile for XSD // Sparx Systems. URL: http://www.sparxsystems.com/uml_tool_guide/xml_technologies/uml_profile_for_xsd.htm (accessed September 7, 2022).

APPENDIX A

Course Project Topic Options

1. Personal Library. Home library catalog: book details (authors, title, publisher, etc.), library category (specialized literature, hobbies, home economics, fiction, etc.), origin and current availability of the book, subjective evaluation of the book.

Selecting books at random; taking inventory of the library.

2. Interpol's card index. Data on each registered criminal: last name, first name, alias, height, hair and eye color, distinguishing features, citizenship, place and date of birth, last place of residence, language skills, criminal profession, most recent case, etc. Criminal and mafia groups (data on accomplices).

Filtering by any subset of attributes. Transfer of those who have reformed to the archive; deletion—only after death.

3. Dating agency. Database of potential partners: gender, registration number, date of registration, personal information, requirements for a partner. Selection of a subset of suitable candidates, arranging meetings (generating invitations for introductions).

Transferring couples who have resolved their family issues to the archive; deleting clients who have declined services.

4. Job placement agency. Database of unemployed individuals: personal information, occupation, education, location and position of last job, reason for termination, marital status, living conditions, contact information, requirements for future employment. Database of job openings: company, position, working conditions and pay, living conditions, requirements for the candidate. Searching for and registering options on both sides, generating print-ready ads, archiving records after employment, and permanently deleting records upon cancellation of services.

5. Address book. Personal information, addresses, phone numbers, place of work or study, job titles of acquaintances, colleagues, and relatives, nature of the relationship, professional qualities, etc. Automatic generation of birthday greetings (based on the current date).

Sorting by alphabetical order and by date of last update.
Search by any criteria.

6. Airport ticket counter. Schedule: flight number, route, layover locations, departure time, flight dates. Number of available seats on each flight. Selection of the nearest flight to a specified destination (subject to availability), issuance of a specified number of tickets in consultation with the passenger (with a corresponding reduction in the number of available seats), and preparation of the boarding pass.

7. Consumer Guide (Consumer Services). Database of the city's consumer service businesses: name, category, address and phone numbers, specialization, list of services provided, form of ownership, hours and days

Search for businesses by a specified service or other criteria.

8. Shopper's Guide. Database of retail outlets in the city: name, address, and phone numbers; specialization; ownership type; and hours of operation. Selection of stores based on any criteria.

9. Single-salesperson store. A computer instead of a cash register. Inventory database: item name, unit of measure, unit price, quantity, date of last restock. Recording of incoming goods (both existing and new items). Processing a purchase: issuing a receipt, updating the inventory database.

Issues with markdowns and write-offs. Inventory of remaining stock with calculation of total value.

10. Human Resources Department. Database of company employees: personal information, education, specialty, department, position, salary, dates of hire and last assignment, etc.

Search by custom criteria. Workforce reduction: selection of employees of retirement and pre-retirement age for termination, preparation of an order.

11. Family tree. Personal information for members of a specific family clan; links to children (or parents). Search for all descendants or all ancestors of a specified person.

12. Warehouse. Database of goods stored in the warehouse: name, unit of measure, unit price, quantity, date of last delivery. Registration of goods received (creation of a goods receipt) and shipped (goods issue). Generation of an inventory report.

13. Bus Station Ticket Office. Bus schedule: route number, terminal and intermediate stops, departure time. Number of available seats on each route. Selection of the nearest route to a specified destination (if seats are available), ticket issuance, and boarding list preparation. Advance ticket sales and ticket refunds.

14. Hotel administrator. Room list: room category, number of beds. Guest list: passport information, arrival and departure dates, room number. Guest check-in: selecting an appropriate room (subject to availability), registration, and issuing a receipt. Check-out: selecting all guests departing today, checking them out, or processing an extension with the issuance of an additional receipt. Option for early check-out with a refund. Searching for a guest by any criterion.

15. Music Lover's Guide. Database of bands and artists; song database; album database with track lists (as links). Select all songs by a specified band; all albums containing a specified song.

16. Diary. Database of scheduled events—date, time, duration, and location. Automatic reminders for the next task: based on the current date and time; option to delete yesterday's tasks or reschedule them for the future. Analysis of "overlaps"—scheduling conflicts between planned tasks. View tasks for tomorrow, the day after tomorrow, and so on.

17. Terminology. A database of definitions for any field of study: enter a term, its interpretation (definition), and references to related terms. Ability to view the entire chain from a given term back to its primary concepts.

18. Head Chef. Database of recipes: ingredient list, preparation instructions. Database of inventory items: name, price, quantity. Creating a daily menu (for a specified number of people); planning a dinner party. Checking inventory levels: generating a warehouse withdrawal slip, adjusting inventory levels.

19. Physician's Handbook. Disease database: name, symptoms, procedures, list of recommended medications with specified quantities. Inventory database: name, quantity, interchangeability. Generating prescriptions after patient examinations, checking medication availability, and adjusting inventory levels.

20. Admission of Applicants. Applicant database; personal information, aggregate scores on entrance exams, willingness to study on a contract basis. Selection of a specified number of applicants for admission; generation of a list for interviews of those who have achieved the minimum passing score but cannot pay for their education.

21. Company Directory. Name, address, and phone numbers; CEO; status (ownership structure); raw materials; products. Selection based on a custom template.

22. Housing Exchange. Database of exchange offers: neighborhood, square footage, floor plan, etc.; requirements for exchange options. Client registration, selection of suitable options, removal upon exchange or rejection.

23. Prospective Student Directory. Database of higher education institutions: name, address, list of majors, last year's admission competition for each major (full-time, evening, and distance learning), tuition fees for contract-based programs. Search by various criteria: everything about this higher education institution; everything about this major; search for the lowest admission competition for a specific major or overall.

24. Postal Code Directory. **Country**, region, district, locality, postal code. Search by any combination of fields (except the last one); hierarchical relationships between fields (note that, for example, **Yampil is located in the Vinnytsia, Khmelnytskyi, and Donetsk regions**).

25. Oschadbank. Information about the bank's depositors: personal account number, deposit category, passport details, current deposit balance, date of the last transaction. Transactions involving deposits and withdrawals of any amount, automatic interest accrual.

26. Pawnshop. Database of pledged goods and real estate: customer profile information, item description, appraised value; amount loaned against the collateral, date of deposit, storage period. Transactions involving the acceptance, return, and sale of goods upon expiration of the storage period.

27. Breeder's Reference Guide. Name of the variety of any crop, breeder, parent varieties, yield, fruit characteristics, frost resistance, resistance to pests and diseases, availability in a

collection. Selection of varieties possessing specified characteristics.

28. Video Library Owner's Reference Guide. Video database: title, studio, genre, year of release, director, lead actors, synopsis, subject-based film rating. Indication of whether the film is available in the video library. Processing of cassette check-outs and returns.

29. Buy and Sell. Seller database: product name, batch size for wholesale, price, terms of sale—shipping, payment method, contact address or phone number, note (for example, “No middlemen, please”). Buyer database: product name, purchase quantity, acceptable price and payment method, contact address or phone number, note. Searching for and registering options from both sides, generating print-ready listings, archiving listings after a sale (as one of the clients may remain dissatisfied), and complete removal upon refusal of services.

30. Fan Reference Guide. Athlete database: personal and anthropometric data, citizenship, origin, sport, club or team, information on personal records or victories, etc. Selection by any criterion. Search for a record-holder in a specified sport.

31. Amateur Radio Reference Guide. Reference databases for transistors, diodes, thyristors, etc.: brand, specifications, maximum operating limits, price, interchangeability information, etc. Selection based on specified requirements.

32. Commercial Banks Reference Guide. Name, address, status (ownership structure), terms for holding funds in a personal account (annual interest rates for various types of deposits). Select the bank offering the highest interest rate for a given type of deposit.

33. Prison Warden's Directory. Personal information on inmates, charge, sentence length, date of arrest, position in the prison hierarchy, cell, information about relatives, personality traits. Generation of statistical reports on the inmate population; selection based on any criterion.

34. Commander's Directory. List of subordinate military personnel: personal information, parents' address, civilian occupation, education, rank and date of promotion, position, unit, branch of service, personality traits, and attitude toward service. Generating lists: of a specific unit, officers, recruits, etc.

35. Real Estate Agency (buying and selling homes). List of listings: neighborhood and address, description of the house or apartment, asking price, applicant's contact information. Demand database: buyer's housing requirements (possibly multiple options, acceptable ranges), financial capacity, and applicant's contact information. Selection of options for one party or the other; automated search for mutually acceptable options. Example of a buyer's request: one-bedroom apartment, up to 100,000 UAH, do not offer properties in Inkerman.

36. Housing waiting list. List of applicants for housing allocation and upgrades: date of registration with the agency, date of application submission, family composition, housing

conditions as of the application date, eligibility for additional space, out-of-turn and priority upgrades, preferred neighborhoods, etc. Allocation of newly available municipal housing: satisfying eligible applicants on the waiting list, relocating subsequent applicants to vacated apartments. Adding applicants and removing those who have dropped out or been satisfied.

37. Car dealership. Database of new and used domestic and foreign cars: make, model year, technical specifications, features, condition, asking price. Buyer database: contact information, requirements regarding make, technical specifications, and technical condition, financial capabilities. Automated selection of options for buyers, generation of requests for suppliers and car transporters.

38. Tourist Guide. Travel agencies and services offered: country, city (or cruise route), accommodations and transportation, guided tours, host services, and package price.

39. “Buy and Sell.” A database of classified ads (radio, newspaper, and ticker ads) across all categories. Search for... (e.g., “tutoring & math”).

40. Winged Phrases. A reference guide to proverbs, sayings, aphorisms, puns, and other linguistic curiosities. Classified by author and source; searchable by topic and keyword.

41. Car Parts Catalog. A car consists of several thousand parts; some are used across different makes. Tables: country, manufacturer, car make, assembly, unit, part. Interchangeability information. Users: auto service employees, auto parts store staff, and wholesale suppliers.

42. Radio Parts Catalog (TV Equipment Reference Guide). Models of consumer audio and video equipment; for each model—a catalog of the radio parts used in it (resistors, capacitors, inductors, chips, etc.). Many parts are used in different models; some parts are interchangeable in one direction (for example, a 10K 0.25 W resistor can be substituted, but not vice versa) or in both directions (transistors with similar characteristics). Tracking parts inventory and searching for suitable parts for repairs.

43. Numismatist’s Reference Guide. Coins: country, denomination, year of issue, mintage, distinctive features. Collectors: country, name, contact information, presence of rare coins in the collection. Personal collection.

44. Philatelist’s Reference Guide. Stamps: country, face value, year of issue, print run, distinctive features. Philatelists: country, name, contact information, presence of rare stamps in the collection. Personal collection.

45. Art Lover’s Guide. Artists with biographical information and artistic styles. Paintings with references to the artists, creation dates, and genres. Collectors and museums: whether they possess original paintings or copies. Auctions and consignment shops: dates, lists of featured masterpieces, and prices

. My Collection.

46. **Tak-Tikli**. This game is similar to checkers and tic-tac-toe. The game board consists of 16 squares (4x4). Each player places their four pieces on opposite sides of the board, alternating with the opponent's pieces, as shown in the illustration. On a single turn, you can move one of your pieces one empty square up or down, right or left, but not diagonally. You cannot capture your opponent's pieces. The objective is to line up three pieces of your color in a row—vertically, horizontally, or diagonally—faster than your opponent.

47. **Knight's Move**. Objective of the game: to score more points than your opponent. Rules of the game: players take turns moving the knight. The direction of the move is arbitrary but must follow the rules of a chess knight's movement. The number indicated on the square where the player placed the piece is added to the player's score, and the turn passes to the opponent. A player moves from the square where the opponent placed a piece.

48. **Bishop's Move**. Objective: to score more points than the opponent. Rules of the game: Players take turns moving the bishop. The direction of the move is arbitrary but must follow the rules for a chess bishop's movement. The number indicated on the square where the player placed the piece is added to the player's score, and the turn passes to the opponent. A player moves from the square where the opponent placed a piece.

49. **The Queen's Move**. Objective: to score more points than the opponent. Rules of the game: Players take turns moving the queen. The direction of the move is arbitrary but must follow the rules for the movement of a chess queen. The number indicated on the square where the player placed the piece is added to the player's score, and the turn passes to the opponent. A player moves from the square where the opponent placed a piece.

50. **Rook's move**. Objective of the game: to score more points than the opponent. Rules of the game: Players take turns moving the rook. The direction of the move is arbitrary but must follow the rules of chess rook movement. The number indicated on the square where the player placed the piece is added to the player's score, and the turn passes to the opponent. A player moves from the square where the opponent placed a piece.

51. **The Amazon's Move**. Objective of the game: to score more points than your opponent. Rules of the game: The Amazon is an imaginary <chess> piece that combines the characteristics of the queen and the knight. Players take turns moving the Amazon. The direction of the move is arbitrary but must comply with the specified rules. The number indicated on the square where the player placed the piece is added to the player's score, and the turn passes to the opponent. The player moves from the square where the opponent placed their piece.

52. **Hangman**. The goal of the game is to guess the word chosen by the computer. Game rules: You can choose any letter. If that letter is in the word, all instances of that letter in the word will be revealed.

53. **Grid**. The game board is 10x10 with randomly distributed

numbers inside them. The goal of the game is to score more points than the opponent. Game rules: The first player moves horizontally, and the second player moves vertically. The first player may select any cell containing a number on the current horizontal row. If the cell is red, the number of points indicated on it is added to the player's score; if it is blue, the points are subtracted. After the first player's move, the vertical column containing the cell they selected becomes the current one, and the turn passes to the opponent, who can select any cell with a number on the current vertical column, and so on.

54. Diagonals. The game board is 6x6. The objective of the game is to occupy as much space on the board as possible. Rules: When it's your turn, you may place one piece on any cell on the board. If, after your move, one or more of your opponent's pieces end up between your pieces, all of those pieces become yours.

55. Double-dealing fellows. The game board is 6x6. The goal of the game is to occupy more space on the board than your opponent. Rules: You can move your piece to the next square on the board; when you do, the piece splits into two—that is, it appears in the new position and remains in the old one at the same time. You can also jump over one square, but in this case, the piece does not split. If adjacent squares are occupied by your opponent, they become yours.

56. Senegalese Checkers. The board size is 5x6 squares. Each player receives 14 checkers (shells and stones), which are arranged so that the two outer top and bottom rows are filled with checkers of the corresponding color. In the center, the checkers are placed from the edges so that two empty squares are formed, from which the game begins. You can only move to an empty square in front, to the right, or to the left, one square per move. You cannot move backward or diagonally. If an opportunity arises to jump over an opponent's checker, the player must do so. In this case, the opponent's checker is considered "captured" and is removed from the board. The game ends when one player's checkers have advanced so far that they can no longer "capture" any more checkers. The number of captured checkers is then counted, and the winner is determined.

57. Japanese Checkers. The board measures 10x10. The objective of the game is to place 5 pieces in a row (vertically, horizontally, or diagonally). Rules of the game: A piece may be placed on any empty space on the board.

58. Corners. The board is 7x7. Initially, the pieces are arranged in two 3x3 blocks in opposite corners of the board. The objective of the game is to move all pieces to the opposite corner. A piece can be moved in two ways: one square to the left, right, up, or down, or by jumping over other pieces.

59. Blockade. The game board is 7x7. Initially, the pieces are arranged in two 3x3 blocks in opposite corners of the board. The goal of the game is to move all the pieces to the opposite corner faster than your opponent. Game rules: You can move a piece up to three squares at a time through empty adjacent squares

a turn; you may jump over a piece. In this case, the piece that the player jumped over is sent “home”—to the corner from which it began its move. Each player may make no more than 10 jumps per game.

Note: Players may choose their own theme. In this case, the theme must be agreed upon with the instructor.

APPENDIX B
Example of a Title Page Format

Ministry of Education and Science of Ukraine
Vinnytsia National Technical University
Department of Software Engineering

COURSE PAPER
in the course **“Object-Oriented Programming”**
on the topic: DEVELOPMENT OF A SOFTWARE SYSTEM FOR MODELING
OBJECTS “MUSICIANS, HOTELS, CITY”
USING UML AND THE JAVA PROGRAMMING LANGUAGE

Student__ , __th year, group __PI-b, major 121
“Software Engineering,” A. I. Kopeikina

Advisor: Associate Professor of the Department of Software
Engineering, Ph.D. in Technical Sciences, D. I. Katelnikov
National Scale _____
Number of points: _____ Grade: ECTS _____

Committee
Members

(signature)

(Last Name and Initials)

(signature)

(Last Name and Initials)

Vinnytsia – 2023

APPENDIX B

Example of an Individual Assignment Format

Ministry of Education and Science of Ukraine
Vinnytsia National Technical University
Faculty of Information Technology and Computer Engineering

APPROVED
Head of the Department of Software
Engineering, Professor, Doctor of
Technical Sciences

” ” _____ Year
O. N. Romanyuk

INDIVIDUAL ASSIGNMENT

for a term paper in the course “Object-Oriented Programming” by student A. I. Kopeikin of
group 1PI-22b

DEVELOPMENT OF A SOFTWARE SYSTEM FOR MODELING THE OBJECTS “MUSICIANS, HOTELS,
CITY” USING UML AND THE JAVA PROGRAMMING LANGUAGE

Initial Data:

- Development environments: NetBeans IDE, IntelliJ IDEA, or Eclipse; JavaFX platform.

Course Project Requirements:

The program must implement the following: a Windows application with a graphical user interface. The micro-object class must contain at least 4 variable elements and at least 4 methods (excluding getters and setters). Constructor delegation must be used. At least one static initialization block and one non-static initialization block must be demonstrated. The boolean equals(Object o) and String toString() methods must be implemented. Deep copying must be implemented by implementing the Cloneable interface. The use of the Comparable or Comparer interface must be demonstrated. The use of virtual methods must be demonstrated. The 'protected' access modifier must be demonstrated. The use of dynamic and static polymorphism must be demonstrated. Demonstrate the use of the 'instanceof' keyword. Micro-object: at least three graphical primitives (one of which is text). Universal object: at least three graphical primitives (one of which is text). Macro-object: at least three graphical primitives (one of which is text). When the left mouse button is clicked on a micro-object, it must become active or inactive. The appearance of an active micro-object must differ from that of an inactive one by at least one graphical primitive. The program interface must indicate which micro-object the user has activated. At the student's discretion, information about the activated micro-object(s) may be displayed in the status bar or the program's workspace. When the arrow keys are pressed, the active object(s) must move within the program window. When the Delete key is pressed, the active objects must be deleted. If there is no active object, the Delete key is ignored. When the Esc key is pressed, the activation of micro-objects must be canceled. A hierarchy of micro-object classes must be implemented, containing at least three levels of inheritance. When the Insert key is pressed, a dialog box should appear to define the parameters of the micro-object being created. It must include the option to select which of the descendant classes in the inheritance hierarchy the micro-object belongs to. In addition to the Button control, the dialog box must also include at least three of the following: TextField, CheckBox, {ComboBox|ListView|ChoiceBox}, RadioButton. Add a universal object class or classes to the project. The universal object is designed to contain all micro-objects and all macro-objects. Only a specific part of the universal object is displayed in the application window, which must not exceed 25% of its total size. Add macro-object classes to the project. A macro-object must be able to contain more than one micro-object. Micro-objects can either belong to one (or more) macro-objects or belong to none. The ability to remove a micro-object from all macro-objects must be implemented. The ability to manually place a micro-object into a macro-object must also be implemented. Automatic movement of certain micro-objects must be implemented: 1) during movement, some micro-objects interact with each other; 2) some micro-objects enter macro-objects and some exit them; 3) the nature of the micro-objects' movement must change when a specific keyboard key is pressed or a menu command is selected. Serialization/deserialization of all objects into a text/binary/XML file must be programmed. The use of a SINGLE parameterized collection is demonstrated.

Software Contents for the Course Project

Individual Assignment

ABSTRACT

INTRODUCTION

1 ANALYSIS OF THE CURRENT STATE OF THE ISSUE AND JUSTIFICATION OF THE PROJECT TASK

2 DEVELOPMENT OF THE PROGRAM INTERFACE AND USAGE EXAMPLES

3 DEVELOPMENT OF THE GRAPHICAL DISPLAY SUBSYSTEM

4 DEVELOPMENT OF CLASS, OBJECT, AND STATE DIAGRAMS

5 USE OF JAVA FX PROGRAMMING TOOLS

6 DEVELOPMENT OF THE DATA SERIALIZATION/DESERIALIZATION SUBSYSTEM

7 USER MANUAL CONCLUSIONS

REFERENCES

Appendices (if necessary)

Date of Publication _____ Year

Supervisor

Task received by _____

APPENDIX G

Use of Algorithm Diagrams, Programs, and Data

Algorithm diagrams, programs, data, and systems consist of symbols with specified meanings, brief explanatory text, and connecting lines. All symbols are divided into the following subgroups:

- data symbols (Table G.1);
- process symbols (Table G.2). An example is shown in Fig. G.1;
- special symbols (Table G.3). An example is shown in Fig. G.2;
- line symbols (Table G.4). An example is shown in Fig. G.3.

Symbols are divided into:

- basic symbols, used when the exact form of the process or data carrier is unknown, or when there is no need to describe the actual data carrier;
- specific symbols, used when the exact nature of the process or data carrier is known, or when it is necessary to describe the actual data carrier.

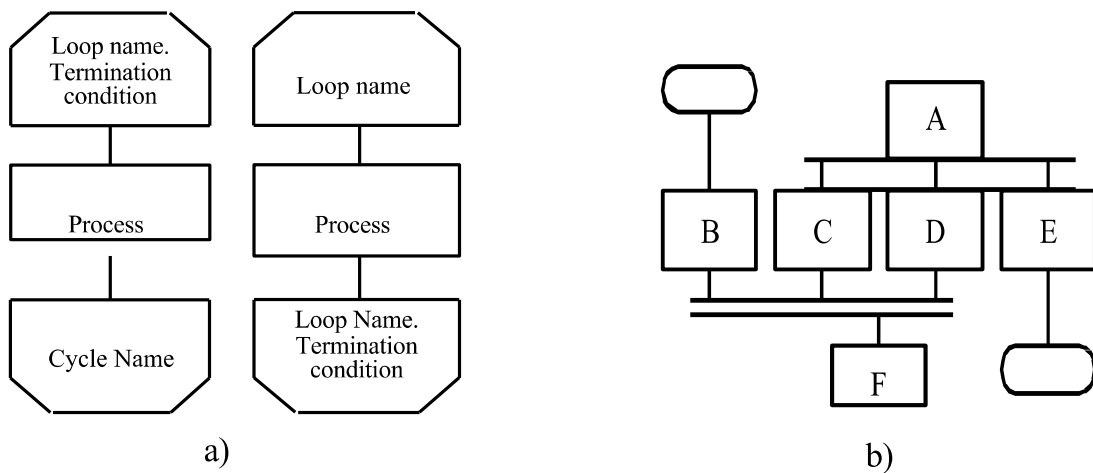


Figure G.1 – Example of the use of process symbols (a – use of cycle boundary symbols; b – use of parallel action symbols)

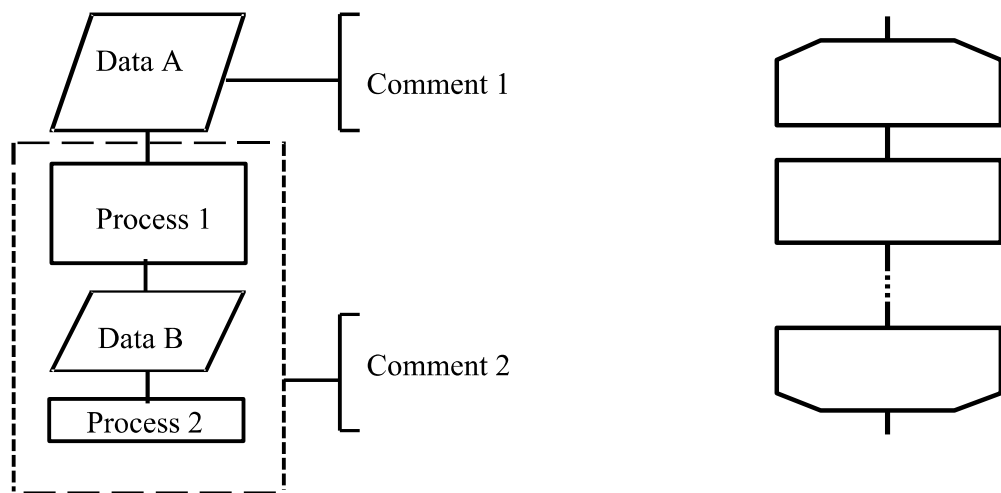


Figure D.2 – Examples of special character usage

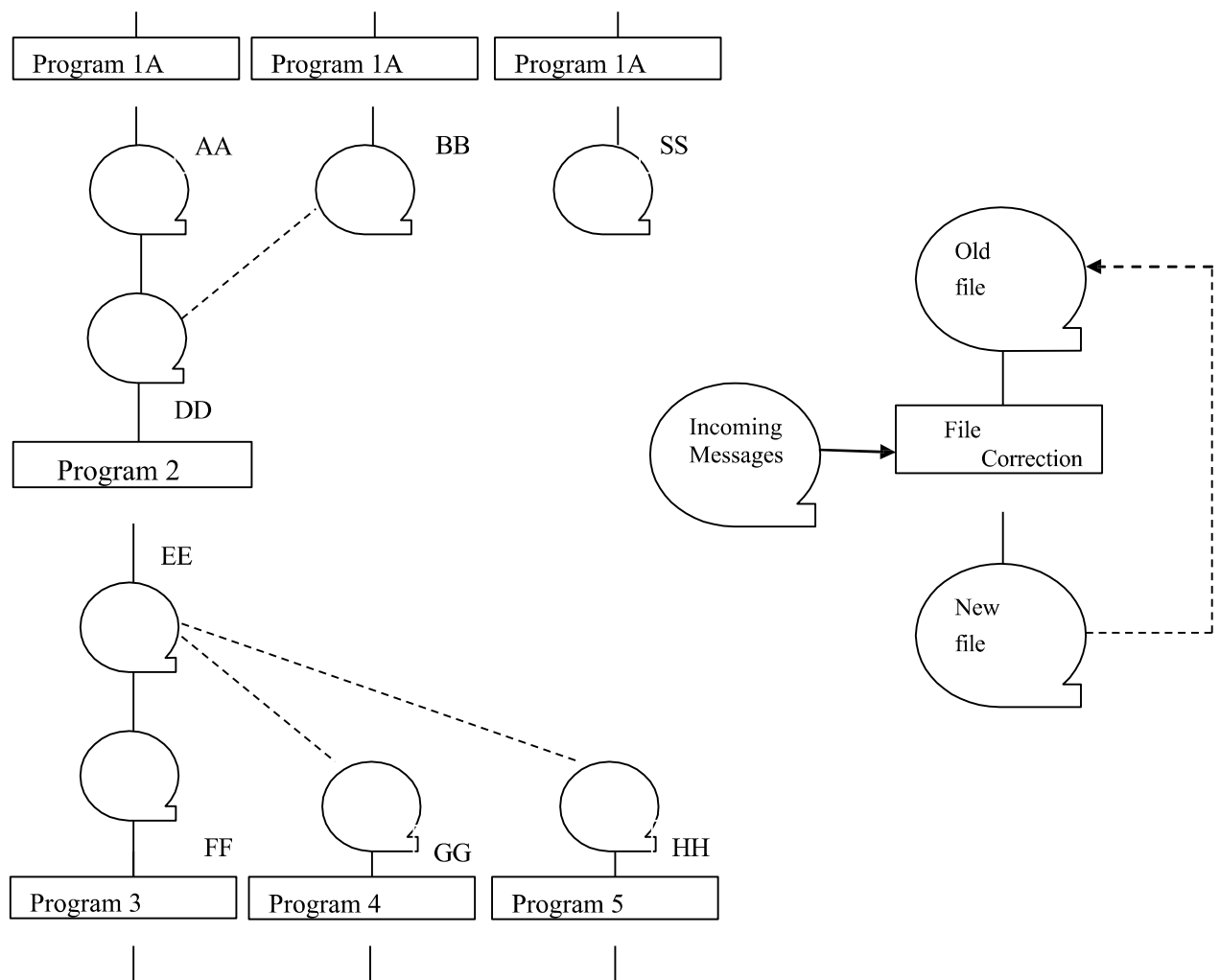


Figure G.3 – Examples of Line Symbol Usage

Data flow diagrams depict the path of data during problem-solving and define the processing stages, as well as the various data storage media used. A data flow diagram consists of the following symbols: data symbols, which may also indicate the type of data medium; process symbols, which represent operations performed on the data (and may also indicate functions performed by a computer); line symbols, which indicate data flows between processes and/or data media; and special symbols to facilitate the writing and reading of the diagram.

Data symbols alternate with process symbols. A data diagram begins and ends with data symbols (with the exception of special symbols).

Program flowcharts depict the sequence of operations in a program. A program flowchart consists of: process symbols indicating actual data processing operations (including symbols that define the path to be followed based on logical conditions); line symbols indicating the control flow; and special symbols used to facilitate the writing and reading of the flowchart.

System flowcharts depict the control of operations and the flow of data within the system. A system flowchart consists of: data symbols indicating the presence of data (data symbols may also indicate the type of data medium); process symbols indicating the operations to be performed on the data, as well as defining the logical path to be followed; line symbols indicating data flows between processes and/or data carriers, as well as control flows between processes; and special symbols used to facilitate the creation and reading of the flowchart.

A program interaction diagram depicts the path of program activations and interactions with the corresponding data. Each program appears only once in a program interaction diagram (in a system flowchart, a program may appear in more than one control flow). A program interaction diagram consists of: data symbols indicating the presence of data; process symbols indicating the operations performed on the data; line symbols representing the flow between processes and data, as well as the initiation of processes; and special symbols—to facilitate the writing and reading of the diagram.

A system resource diagram depicts the configuration of data blocks and the processing units for these data blocks required to solve a problem or a set of problems. A system resource diagram consists of: data symbols representing the input, output, and storage devices of a computer; process symbols representing processors, channels, etc.; line symbols representing data transfer between input/output devices and processors, as well as control transfer between processors; and special symbols used to facilitate the writing and reading of the diagram.

Table G.1 – Data Symbols

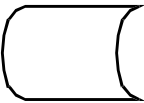
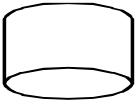
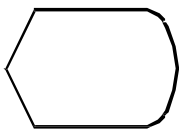
<i>Basic Data Symbols</i>		
Data		The symbol represents data; the data carrier is unspecified
Stored Data		The symbol represents stored data in a form suitable for processing; the data carrier is undefined
<i>Specific data symbols</i>		
Random-access storage device		The symbol represents data stored in a random-access memory
Storage with sequential access		The symbol represents data stored in a sequential-access storage device (magnetic tape, magnetic tape cassette, audio cassette)
Storage direct-access storage device		The symbol represents data stored in a random-access storage device (magnetic disk, etc.)
Document		This symbol represents data stored on a medium in a format readable form (optically or magnetically scannable document, microfilm, data forms)
Manual entry		The symbol represents data entered manually during processing from any device (keyboard, switches, buttons, barcode strips)
Display		The symbol represents data presented in a visual, human-readable form on a medium in the form of a display device (visual display screen, information input indicators)

Table G. 2 – Process Symbols

<i>Basic process symbols</i>		
Process		This symbol represents a data processing function of any kind (performing a specific operation or a group of operations that results in a change in the value or form of the information).
<i>Process-specific symbols</i>		
Subordinate process		The symbol represents a subordinate process consisting of one or more operations or program steps that are defined elsewhere

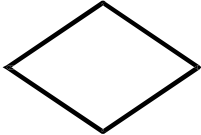
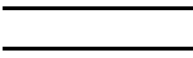
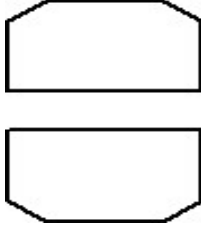
Manual operation		This symbol represents any process performed by a person
Preparation		The symbol represents the modification of a command or group of commands in order to affect some subsequent function (setting a switch, modifying an index register, or program initialization)
Condition or selection		The symbol represents a condition, a choice, or a switch-type function that has one input and a number of alternative outputs, only one of which can be activated after the conditions defined within this symbol
Parallel Operations		The symbol represents the synchronization of two or more parallel operations
Cycle Boundary		A symbol consisting of two parts represents the start and end of a loop. Both parts of the symbol have the same identifier. The conditions for initialization, increment, and termination are placed inside the symbol at the beginning or end, depending on the location of the operation that checks the condition

Table G. 3 – Special Symbols

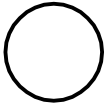
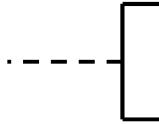
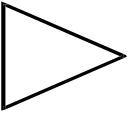
Connector 	The symbol represents an output from one part of a circuit and an input from another part of that circuit, and is used to break a line and continue it elsewhere. Corresponding connector symbols must have the same unique designation
Terminator 	The symbol represents an output to the external environment and an input from the external environment (the beginning or end of a program circuit, external usage, and the source or destination)
Comment 	This symbol is used to add descriptive comments or explanatory notes for clarification or annotation. The dotted lines in the comment symbol are connected to the corresponding symbol or may outline a group of symbols. The text of comments or notes must be placed adjacent to the enclosing shape
Omission 	The symbol (ellipsis) is used in diagrams to represent an omission of a symbol or group of symbols for which neither the type nor the number of symbols is specified. The symbol is used only within or between line symbols. It is primarily used in diagrams depicting general results of a selection with an unknown number of repetitions

Table G. 4 – Line Symbols

<i>Basic line symbol</i>	
Line 	The symbol represents a data or control flow. If necessary, or to improve readability, arrows may be added
<i>Specific line symbols</i>	
Control Transfer 	The symbol represents the direct transfer of control from one process to another, sometimes with the option of returning directly to the initiating process after the initiated process completes its functions. The type of control transfer must be specified within the symbol (e.g., request, call, event)
Communication channel 	The symbol represents data transfer over a communication channel
Dotted line 	The symbol represents an alternative connection between two or more symbols and is also used to outlining an area

APPENDIX D

Examples of UML Diagrams



Figure D.1 – Example of a class diagram

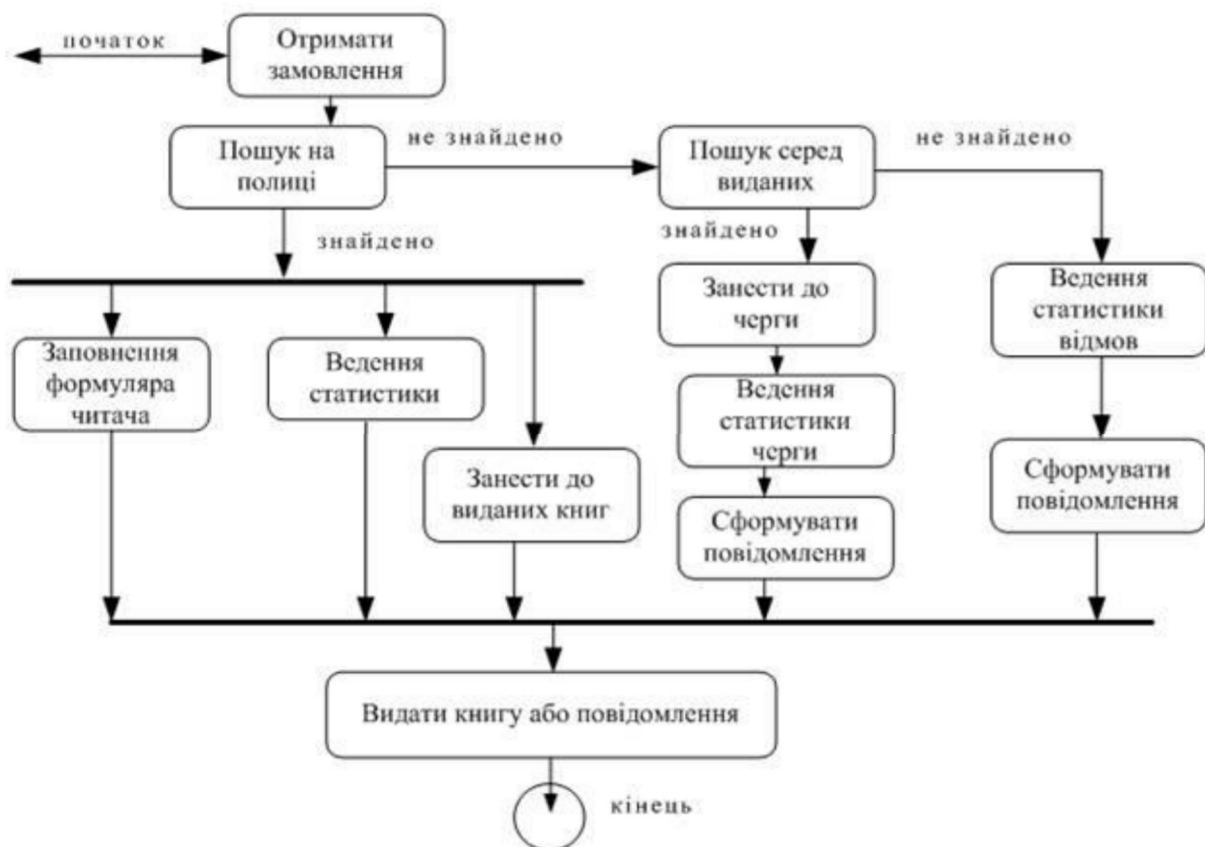


Figure D.2 – Example of an activity diagram

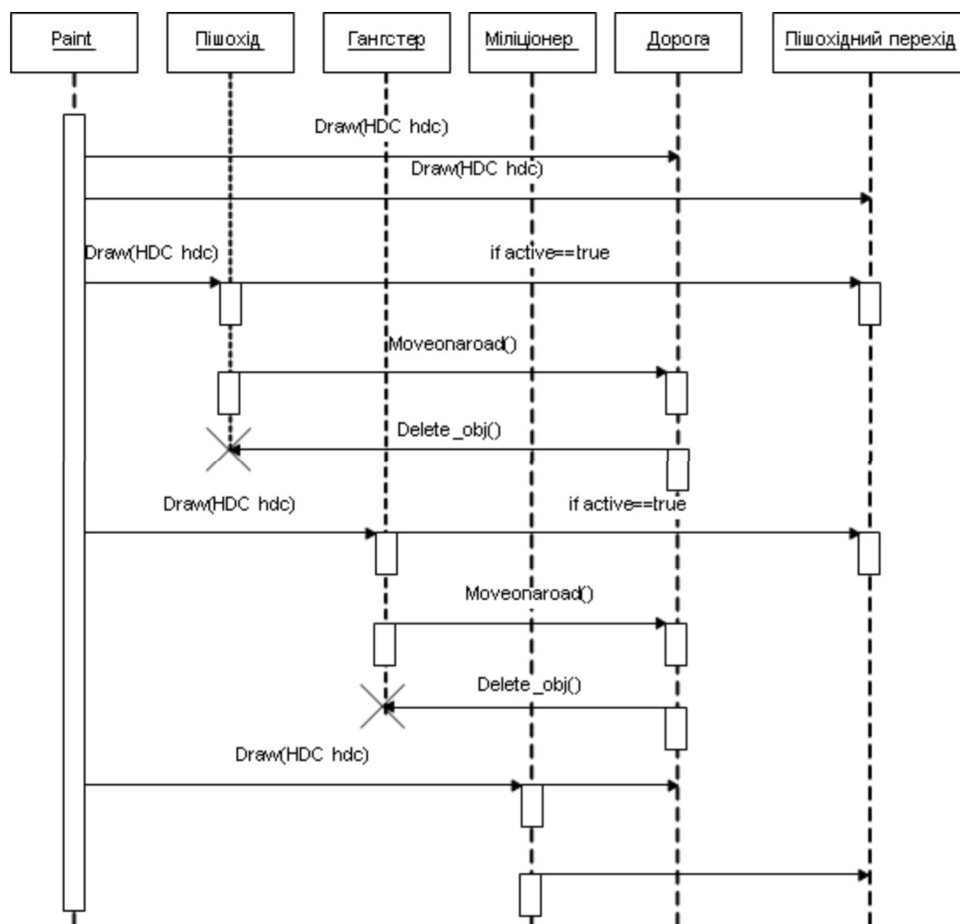


Figure D.3 – Example of a sequence diagram

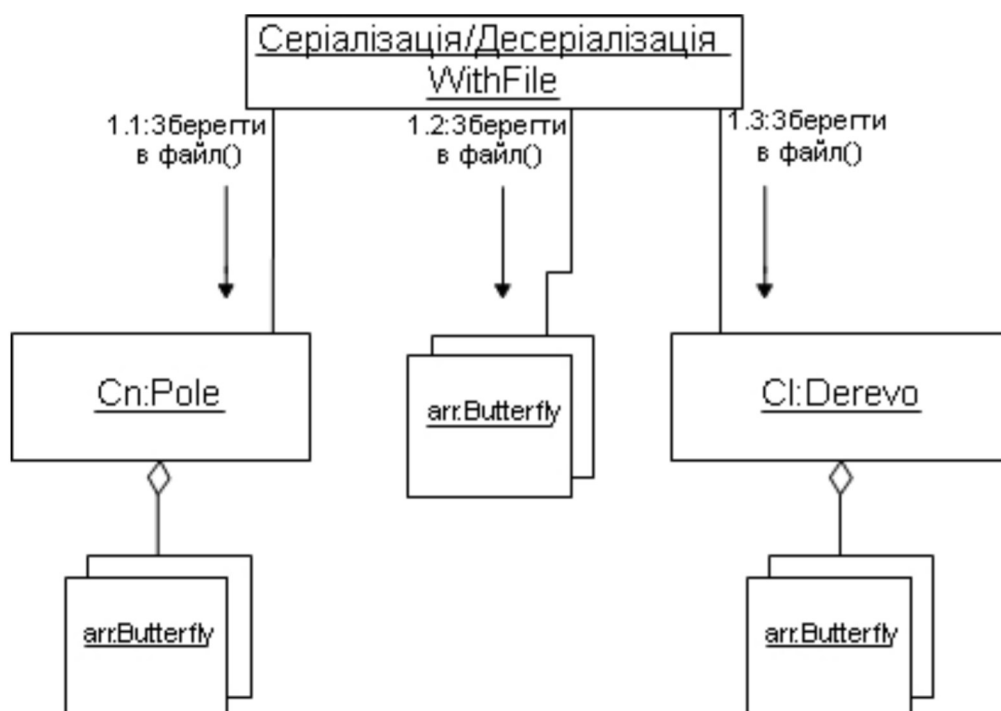


Figure D.4 – Example of a collaboration diagram

Instructions
for completing course assignments
in the course “Object-Oriented Programming” for students
majoring in
121 “Software Engineering”

Compiled by: *Denis Ivanovich Katelnikov,*
Alla Vasylivna Denisyuk

Manuscript formatted by *D. I. Katelnikov*

Editor: *K. Dedova*

Original layout prepared by

Approved for printing on __ 2023

Format: 29.7×42¼. Offset paper.

Font: Times New Roman.

Risographic printing. Standard printed sheets:

__ Print run: 40 (1st run: 1–21) copies. Order No.
2023-_____

Publisher and producer: Vinnytsia
National Technical University,
Editorial and Publishing Department.

VNTU, GNK, Room 114.

95 Khmelnytskyi Highway,

Vinnytsia, 21021.

Tel. (0432) 65-18-06.

press.vntu.edu.ua;

email:kivc.vntu@gmail.com .

Publishing Entity Certificate, Series DK No.
3516, dated July 1, 2009.